DATAMINERS

# Métricas de software
# y comparativa de métricas

SGRSI — Sistema de Gestión de Recursos y Soporte de Informática

## Ingeniería de Software

Proyecto de egreso 2026


**Equipo DataMiners**

Nicolas Pereira — Coordinador
Bruno Coirolo — Sub-Coordinador
Francisco Camacho
Alvaro Larrosa


**Docente: Rodriguez, Gerardo**

**TERCERA ENTREGA**

Fecha de entrega: 23 de octubre de 2026

# Índice

**Índice** ..... **1**

**1. Introducción y alcance** ..... **2**

1.1 Criterios de medición ..... 2

**2. Métricas de software de la versión actual** ..... **3**

2.1 Tamaño ..... 3

2.2 Complejidad ..... 4

2.3 Documentación en el código ..... 4

2.4 Arquitectura y datos ..... 5

2.5 Actividad del repositorio ..... 5

**3. Estimación del tamaño funcional y del esfuerzo** ..... **5**

3.1 Estimación por líneas de código ..... 6

3.2 Análisis por puntos de función ..... 7

3.3 Estimación por puntos de función ..... 8

3.4 Registro del esfuerzo del equipo ..... 8

**4. Comparativa de métricas de tamaño y función** ..... **9**

4.1 Relación entre líneas de código y puntos de función ..... 9

4.2 Comparación de las estimaciones de esfuerzo ..... 10

**5. Comparativa de métricas entre etapas** ..... **11**

5.1 Hitos definidos ..... 11

5.2 Tabla comparativa ..... 11

5.3 Evolución del tamaño ..... 12

5.4 Churn por commit ..... 13

**6. Análisis de la evolución** ..... **13**

6.1 Del prototipo a la estructura ..... 13

6.2 Del frontend al sistema completo ..... 13

6.3 Cierre del desarrollo ..... 13

6.4 La versión actual ..... 14

6.5 Documentación y estabilidad ..... 14

**7. Hallazgos y recomendaciones** ..... **14**

**8. Referencias Bibliográficas** ..... **16**

# 1. Introducción y alcance

El presente documento reúne las tareas de Ingeniería de Software del proyecto de egreso del Sistema de Gestión de Recursos y Soporte de Informática (SGRSI) que se refieren a las métricas de software: la medición de métricas sobre el código del repositorio, la estimación del tamaño y del esfuerzo del proyecto y la comparación de las métricas de tamaño y función y de las métricas entre las distintas etapas del proyecto.

Las métricas de la versión actual se calcularon sobre el código más reciente del repositorio y las de las etapas anteriores sobre el historial de git, que registra 9 commits entre el 17 de junio y el 24 de agosto de 2026. Cada etapa corresponde a una instantánea del código, por lo que todos los valores son reproducibles. Se midieron métricas de producto, como el tamaño y la complejidad del código (Sommerville, 2016), que alimentan características de mantenibilidad del modelo de calidad ISO/IEC 25010 como la modularidad, la analizabilidad y la capacidad de prueba (International Organization for Standardization & International Electrotechnical Commission [ISO/IEC], 2011). Para estimar el esfuerzo se utilizaron dos métodos independientes: el modelo COCOMO básico, basado en las líneas de código (Boehm, 1981), y el análisis por puntos de función, basado en la funcionalidad del sistema (Albrecht, 1979; International Function Point Users Group [IFPUG], 2010).

## 1.1 Criterios de medición

- **LOC (líneas de código):** líneas no vacías y que no son comentarios, en archivos HTML, CSS, JavaScript, PHP y SQL.
- **Comentarios:** líneas de comentario. La densidad es comentarios / (LOC + comentarios).
- **Funciones:** declaraciones `function` y funciones flecha en JavaScript, y declaraciones `function` en PHP.
- **Puntos de decisión:** `if`, `for`, `foreach`, `while`, `case`, `catch`, `elseif`, `&&`, `||` y operador ternario, contados sobre el código sin comentarios ni cadenas de texto.

- **Complejidad ciclomática aproximada:** V(G) = puntos de decisión + 1 por función (McCabe, 1976). Es una aproximación obtenida por análisis de texto y no una medición con una herramienta de análisis estático.
- **Churn:** líneas insertadas y eliminadas según git.
- **Puntos de función (PF):** tamaño funcional según IFPUG (2010). Se cuentan entradas externas (EI), salidas externas (EO), consultas externas (EQ), archivos lógicos internos (ILF) y archivos de interfaz externa (EIF), ponderados por su complejidad. Los PF ajustados resultan de multiplicar los PF sin ajustar por (0,65 + 0,01 × TDI), donde TDI es la suma de los grados de influencia de 14 características generales.

## 2. Métricas de software de la versión actual

| Requerimientos cubiertos — Métricas de software |
| --- |
| • Medición de tamaño, complejidad, documentación y arquitectura sobre el código de la versión actual del repositorio.<br>• Criterios de medición definidos y reproducibles. |

La versión medida es la más reciente del código del proyecto, de octubre de 2026, que se recibió sin historial de commits.

### 2.1 Tamaño

La Tabla 1 presenta el tamaño del código por lenguaje. El 68,3 % del código corresponde al frontend (HTML, CSS y JavaScript, con 2035 LOC) y el 31,7 % al backend y la base de datos (PHP y SQL, con 946 LOC). El tamaño medio es

de 81 LOC por archivo. El archivo más extenso es `layout.css`, con 308 LOC, y el archivo de código más extenso es `inventario.js`, con 197 LOC.

**Tabla 1**

*Tamaño del código por lenguaje (versión actual)*

| Lenguaje | Archivos | LOC |
|---|---|---|
| HTML | 9 | 419 |
| CSS | 4 | 622 |
| JavaScript | 10 | 994 |
| PHP | 12 | 803 |
| SQL | 2 | 143 |
| **Total** | **37** | **2981** |

*Nota.* LOC = líneas de código sin contar líneas vacías ni comentarios. Los 2 archivos SQL son el esquema (`setup.sql`) y un script de actualización (`upgrade_sgrsi.sql`).

## 2.2 Complejidad

La Tabla 2 resume las métricas de complejidad. La complejidad media por función es baja, pero se concentra en los módulos que implementan las reglas de negocio. La Tabla 3 muestra los cinco archivos con más puntos de decisión.

**Tabla 2**

*Métricas de complejidad (JavaScript y PHP)*

| Métrica | Valor |
|---|---|
| Funciones | 140 (JavaScript 110, PHP 30) |
| Puntos de decisión | 384 (JavaScript 219, PHP 165) |
| Complejidad ciclomática aproximada total | 524 |
| Complejidad media por función | 3,74 |

*Nota.* La complejidad ciclomática es una aproximación obtenida por análisis de texto.

**Tabla 3**

*Archivos con más puntos de decisión*

| Archivo | LOC | Puntos de decisión |
|---|---|---|
| `inventario.js` | 197 | 43 |
| `incidencias.js` | 177 | 35 |
| `servicio.js` | 140 | 35 |
| `prestamos.js` | 58 | 34 |
| `api/prestamos.php` | 83 | 28 |

## 2.3 Documentación en el código

El código tiene 105 líneas de comentario frente a 2981 LOC, lo que da una densidad de comentarios del 3,4 %.

## 2.4 Arquitectura y datos

La Tabla 4 resume las características de arquitectura y de datos de la versión actual. El sistema se organiza en tres capas y no cuenta con archivos de pruebas automatizadas.

**Tabla 4**
*Arquitectura y datos de la versión actual*

| Métrica | Valor |
|---|---|
| Capas | Frontend estático (HTML, CSS y JS), API PHP y base de datos MySQL |
| Endpoints de la API | 7: `auth`, `usuarios`, `inventario`, `incidencias`, `solicitudes`, `prestamos` y `dashboard` |
| Operaciones de la API | 33 |
| Modelos | 4: `Usuario`, `Inventario`, `Incidencia` y `Solicitud` |
| Tablas en `setup.sql` | 6: `usuarios`, `ubicaciones`, `inventario`, `incidencias`, `solicitudes` y `prestamos` |
| Pantallas HTML | 9 |
| Archivos de pruebas automatizadas | 0 |

## 2.5 Actividad del repositorio

El historial de git disponible, hasta el commit `225aa01`, registra 9 commits en 6 días distintos, a lo largo de 68 días de calendario (del 17 de junio al 24 de agosto). En total se insertaron 6953 líneas y se eliminaron 4538. La versión actual se recibió sin historial, por lo que no se dispone de su actividad.

# 3. Estimación del tamaño funcional y del esfuerzo

| Requerimientos cubiertos — Estimación de tamaño y esfuerzo |
|---|
| • Estimación del esfuerzo, la duración y el personal del proyecto a partir de las líneas de código y de los puntos de función.<br>• Registro de la información disponible sobre el esfuerzo real del equipo y de sus limitaciones. |

## 3.1 Estimación por líneas de código

El primer método es el modelo COCOMO básico (Boehm, 1981). El modelo calcula el esfuerzo en personas-mes (E) a partir del tamaño en miles de líneas de código (KLOC), la duración en meses (D) y el personal medio (P): E = a × KLOC^b, D = c × E^d y P = E / D. Los coeficientes dependen del modo del proyecto y se presentan en la Tabla 5. Se tomó como referencia el modo orgánico, propio de equipos pequeños que trabajan con requisitos conocidos y flexibles, como el de una aplicación web de uso interno. Los otros dos modos se incluyen como análisis de sensibilidad.

**Tabla 5**

*Coeficientes del modelo COCOMO básico*

| Modo | a | b | c | d |
|---|---|---|---|---|
| Orgánico | 2,4 | 1,05 | 2,5 | 0,38 |
| Semiacoplado | 3,0 | 1,12 | 2,5 | 0,35 |
| Empotrado | 3,6 | 1,20 | 2,5 | 0,32 |

*Nota.* Coeficientes de Boehm (1981).

La Tabla 6 presenta los resultados para las 2981 LOC de la versión actual (2,981 KLOC). La última fila considera solo las líneas de JavaScript y PHP (1,797 KLOC), que son las que implementan la lógica del sistema. Con el modo orgánico se estiman 7,56 personas-mes, una duración de 5,39 meses y un personal medio de 1,40 personas, lo que equivale a unas 1149 horas de trabajo con la convención de 152 horas por persona-mes.

**Tabla 6**

*Estimación de esfuerzo, duración y personal con COCOMO básico*

| Modo (tamaño) | KLOC | Esfuerzo (PM) | Duración (meses) | Personal medio | Horas |
|---|---|---|---|---|---|
| Orgánico | 2,981 | 7,56 | 5,39 | 1,40 | 1149 |
| Semiacoplado | 2,981 | 10,20 | 5,63 | 1,81 | 1550 |
| Empotrado | 2,981 | 13,35 | 5,73 | 2,33 | 2029 |
| Orgánico (solo JS y PHP) | 1,797 | 4,44 | 4,41 | 1,01 | 675 |

*Nota.* PM = personas-mes. Horas = esfuerzo × 152 horas por persona-mes.

## 3.2 Análisis por puntos de función

El segundo método es el análisis por puntos de función con el método IFPUG (IFPUG, 2010), que mide el tamaño del sistema por la funcionalidad que ofrece al usuario y no por su código. El análisis contabiliza 35 funciones: 21 entradas externas (altas, modificaciones, eliminaciones y operaciones de sesión), 1 salida externa (el dashboard de métricas), 7 consultas externas (listados y consulta de sesión) y 6 archivos lógicos internos (usuarios, inventario, incidencias, solicitudes, préstamos y ubicaciones). No hay archivos de interfaz externa porque el sistema no se conecta con otros sistemas. La Tabla 7 presenta el resultado por tipo de función.

**Tabla 7**

*Puntos de función sin ajustar por tipo de función*

| Tipo | Funciones | Complejidad | PF | Implementado | Pendiente |
|---|---|---|---|---|---|
| EI, entrada externa | 21 | 12 baja y 9 media | 72 | 66 | 6 |
| EO, salida externa | 1 | 1 media | 5 | 5 | 0 |
| EQ, consulta externa | 7 | 3 baja y 4 media | 25 | 25 | 0 |
| ILF, archivo lógico interno | 6 | 3 media y 3 baja | 51 | 51 | 0 |
| EIF, archivo de interfaz externa | 0 | — | 0 | 0 | 0 |
| **Total** | **35** | | **153** | **147** | **6** |

*Nota.* Pesos IFPUG: EI 3, 4 y 6; EO 4, 5 y 7; EQ 3, 4 y 6; ILF 7, 10 y 15; EIF 5, 7 y 10 (complejidad baja, media y alta). El estado se verificó contra la versión actual del código. Los 72 PF de entradas externas resultan de sumar el detalle de las 21 funciones.

Según la versión actual del código, el 96 % de los puntos de función ya está implementado (147 de 153). Quedan pendientes dos funciones: la eliminación de usuarios y la modificación de ubicaciones, que suman 6 PF. El análisis no contabiliza cinco operaciones auxiliares que sí existen en el código: el cambio de prioridad de incidencias y de solicitudes, la eliminación de préstamos, el listado de técnicos y el listado de equipos disponibles para préstamo. Si se contaran con complejidad baja o media, el total sin ajustar aumentaría entre 15 y 18 PF.

La Tabla 8 presenta los 14 factores de ajuste. La suma de los grados de influencia es 26, por lo que el factor de ajuste es (0,01 × 26) + 0,65 = 0,91.

**Tabla 8**

*Características generales del sistema y grados de influencia*

| Característica | Grado | Justificación |
|---|---|---|
| 1. Comunicación de datos | 4 | Aplicación web en línea sobre HTTP |
| 2. Procesamiento distribuido | 0 | Sistema no distribuido |
| 3. Requisitos de rendimiento | 1 | Sin requisitos especiales de rendimiento |
| 4. Configuración del equipamiento | 1 | Intranet con restricciones leves |
| 5. Volumen de transacciones | 1 | Entorno académico |
| 6. Entrada de datos en línea | 5 | Más del 30 % de las entradas es en línea |
| 7. Interfaz con el usuario | 4 | Navegación, etiquetas, mensajes en la interfaz y tema |
| 8. Actualización en línea | 3 | La mayoría de los archivos internos se actualiza en línea |
| 9. Procesamiento complejo | 2 | Roles y seguridad con consultas parametrizadas |
| 10. Reusabilidad del código | 1 | Solo dentro de la aplicación |
| 11. Facilidad de implementación | 1 | Script SQL y PHP |
| 12. Facilidad de operación | 1 | Operación normal |
| 13. Instalaciones múltiples | 0 | Una sola instalación |
| 14. Facilidad de cambios | 2 | Listados, filtros y ubicaciones en la base de datos |
| **Suma de grados (TDI)** | **26** | |

## 3.3 Estimación por puntos de función

Los puntos de función ajustados son 153 × 0,91 = 139,2, es decir, 139 PF. Con una productividad supuesta de 8 horas por punto de función, el esfuerzo estimado es de 1112 horas-hombre. Con 3 desarrolladores que trabajan 5 horas por día y 20 días por mes, la duración es de 3,7 meses. El costo se

calculó con un valor de $30.000 por persona-mes y otros costos fijos de $250.000. La Tabla 9 detalla el cálculo.

**Tabla 9**

*Estimación de esfuerzo, duración y costo por puntos de función*

| Concepto | Valor | Cálculo |
|---|---|---|
| Puntos de función sin ajustar (VPFSA) | 153 PF | 72 + 5 + 25 + 51 |
| Suma de grados de influencia (TDI) | 26 | 14 características generales |
| Factor de ajuste | 0,91 | (0,01 × 26) + 0,65 |
| Puntos de función ajustados (VPFA) | 139 PF | 153 × 0,91 = 139,2 |
| Esfuerzo | 1112 horas-hombre | 139 PF × 8 h/PF |
| Duración | 3,7 meses | 1112 h / 3 desarrolladores / 5 h por día / 20 días por mes |
| Costo total | $583.000 | (3 × 3,7 × $30.000) + $250.000 |

*Nota.* Supuestos del análisis: 8 horas por punto de función, 3 desarrolladores, 5 horas por día, 20 días por mes, $30.000 por persona-mes y $250.000 de otros costos fijos.

### 3.4 Registro del esfuerzo del equipo

El equipo no registró horas de trabajo por tarea ni por integrante. Las tareas se asignaban desde Trello y su avance se revisaba en reuniones diarias de 30 minutos (de lunes a viernes de 21:30 a 22:00 y los fines de semana de 18:30 a 19:00). Las 20 actas de la segunda etapa, del 12 de julio al 1 de agosto, equivalen a 40 horas-persona de reunión (20 reuniones × 4 integrantes × 0,5 h), si cada una duró lo pautado. Esa es la única información de tiempo documentada, y no incluye el trabajo de desarrollo. Por eso las estimaciones de las secciones 3.1 y 3.3 no se contrastan con horas reales.

## 4. Comparativa de métricas de tamaño y función

| Requerimientos cubiertos — Comparativa de métricas |
|---|
| • Comparación de las métricas de tamaño (líneas de código) y de función (puntos de función) del sistema.<br>• Comparación de las estimaciones de esfuerzo obtenidas con ambos tipos de métrica. |

## 4.1 Relación entre líneas de código y puntos de función

La Tabla 10 relaciona el tamaño del código con el tamaño funcional. Para la relación se usan los puntos de función sin ajustar que ya están implementados (147), porque el código medido no incluye las funciones pendientes. La referencia es la tabla de factores de conversión de QSM (s. f.), que indica el promedio de líneas de código por punto de función en proyectos terminados: 47 para JavaScript y 21 para SQL.

**Tabla 10**

*Relación entre tamaño y función en la versión actual*

| Métrica | Valor |
|---|---|
| LOC totales | 2981 |
| LOC de lógica (JavaScript y PHP) | 1797 |
| PF sin ajustar analizados | 153 |
| PF sin ajustar implementados | 147 |
| PF ajustados | 139 |
| LOC totales por PF implementado | 20,3 |
| LOC de lógica por PF implementado | 12,2 |
| PF implementados por KLOC | 49,3 |
| Referencia QSM, JavaScript (LOC por PF) | 47 |
| Referencia QSM, SQL (LOC por PF) | 21 |

*Nota.* Las referencias de QSM (s. f.) son promedios de proyectos terminados.

El código actual implementa cada punto de función con unas 12 líneas de lógica, muy por debajo de las 47 líneas por punto de función que indica la referencia para JavaScript. Con esa referencia, 147 PF equivaldrían a unas 6900 líneas, y el código tiene 1797 líneas de lógica, un 26 % de ese valor. La diferencia no implica por sí sola una mayor productividad. Puede deberse a que muchas funciones son operaciones simples de tipo CRUD que reutilizan código común (los modelos del backend y los archivos de diálogos y navegación), a que el análisis cuenta una función por cada operación de la API, y a que la referencia proviene de proyectos industriales con más validaciones y manejo de errores.

## 4.2 Comparación de las estimaciones de esfuerzo

La Tabla 11 compara el esfuerzo estimado con la métrica de tamaño (COCOMO) y con la métrica de función (puntos de función). Con el tamaño

total del código, COCOMO en modo orgánico estima 1149 horas, y el análisis por puntos de función estima 1112 horas, un 3 % menos. Que dos métodos independientes coincidan en un esfuerzo de entre 7,3 y 7,6 personas-mes da más confianza a la estimación. Las duraciones no son comparables, porque COCOMO calcula la duración con una fórmula del modelo y el análisis por puntos de función la calcula con supuestos de dedicación (3 desarrolladores y 5 horas por día).

**Tabla 11**

*Comparación de las estimaciones de esfuerzo*

| Método | Base | Esfuerzo (PM) | Horas | Duración (meses) |
|---|---|---|---|---|
| COCOMO orgánico | 2,981 KLOC | 7,56 | 1149 | 5,39 |
| COCOMO orgánico, solo JS y PHP | 1,797 KLOC | 4,44 | 675 | 4,41 |
| Puntos de función | 139 PF × 8 h/PF | 7,32 | 1112 | 3,7 |

*Nota.* PM = personas-mes (152 horas por persona-mes). La duración de los puntos de función supone 3 desarrolladores y 5 horas por día.

# 5. Comparativa de métricas entre etapas

## 5.1 Hitos definidos

Se definieron seis hitos. Cinco corresponden a commits del historial de git y el sexto es la versión actual del código, que se recibió sin historial. Se describen en la Tabla 12.

**Tabla 12**

*Hitos del proyecto*

| Hito | Fecha | Commit | Descripción |
|---|---|---|---|
| H1 | 17/06/2026 | `919e7e6` | Prototipo inicial en HTML |
| H2 | 22/06/2026 | `c263e8e` | Separación de HTML, CSS y JavaScript |
| H3 | 20/07/2026 | `345ddc9` | Frontend modular de varias páginas |
| H4 | 23/08/2026 | `2d0260d` | Backend PHP y base de datos |
| H5 | 24/08/2026 | `225aa01` | Versión con préstamos y dashboard |
| H6 | Oct. 2026 | Sin historial | Versión actual con ubicaciones, asignación de técnicos y cambio de prioridad |

## 5.2 Tabla comparativa

La Tabla 13 compara las métricas de los seis hitos. Los valores de H1 incluyen copias duplicadas de las páginas en dos carpetas auxiliares (`proyecto2` y `la otra cosa`), que se eliminaron en el commit siguiente.

**Tabla 13**

*Comparativa de métricas por hito*

| Métrica | H1 | H2 | H3 | H4 | H5 | H6 |
|---|---|---|---|---|---|---|
| Archivos | 14 | 3 | 17 | 30 | 32 | 37 |
| LOC totales | 1808 | 575 | 642 | 1906 | 2021 | 2981 |
| LOC HTML | 1545 | 118 | 214 | 389 | 392 | 419 |
| LOC CSS | 263 | 282 | 257 | 350 | 350 | 622 |
| LOC JavaScript | 0 | 175 | 171 | 581 | 581 | 994 |
| LOC PHP | 0 | 0 | 0 | 524 | 636 | 803 |
| LOC SQL | 0 | 0 | 0 | 62 | 62 | 143 |
| Líneas de comentario | 19 | 37 | 30 | 98 | 111 | 105 |
| Densidad de comentarios | 1,0 % | 6,0 % | 4,5 % | 4,9 % | 5,2 % | 3,4 % |
| Funciones | 0 | 16 | 18 | 73 | 73 | 140 |
| Puntos de decisión | 0 | 27 | 29 | 179 | 201 | 384 |
| Complejidad media por función | — | 2,69 | 2,61 | 3,45 | 3,75 | 3,74 |
| LOC medio por archivo | 129 | 192 | 38 | 64 | 63 | 81 |
| Archivo más extenso (LOC) | 611 | 282 | 97 | 140 | 140 | 308 |
| Endpoints de la API | 0 | 0 | 0 | 5 | 7 | 7 |
| Tablas de base de datos | 0 | 0 | 0 | 4 | 4 | 6 |

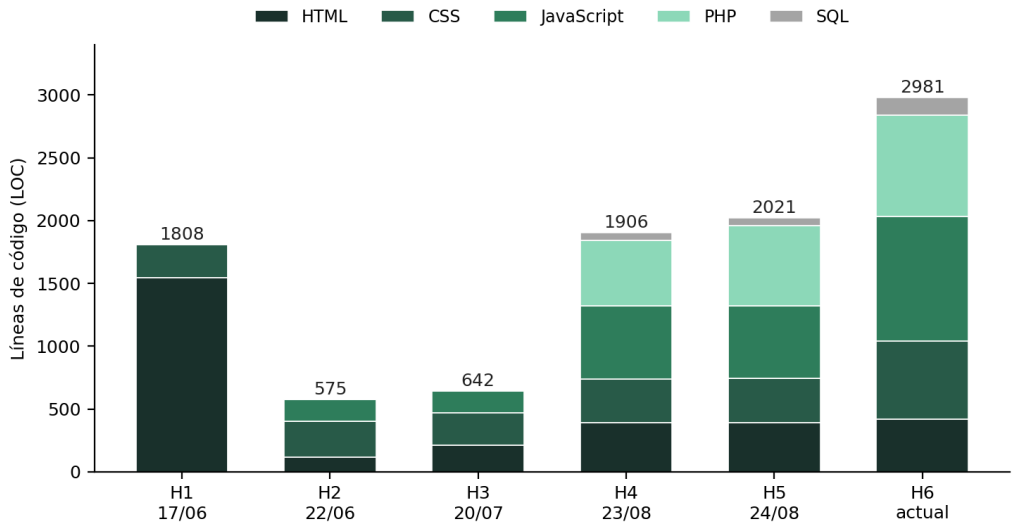
*Nota.* H = hito. Los valores de complejidad se calculan solo sobre JavaScript y PHP.

## 5.3 Evolución del tamaño

La Figura 1 muestra cómo cambió la composición del código por lenguaje a lo largo de los hitos.

**Figura 1**

*Líneas de código por lenguaje en cada hito*




*Nota.* LOC = líneas de código sin contar líneas vacías ni comentarios.

## 5.4 Churn por commit

La Tabla 14 detalla las líneas insertadas y eliminadas en cada commit de los hitos H1 a H5. La versión actual no tiene historial.

**Tabla 14**

*Líneas insertadas y eliminadas por commit*

| Commit | Fecha | Insertadas | Eliminadas |
|---|---|---|---|
| `919e7e6` | 17/06/2026 | 3780 | 0 |
| `5179b71` | 22/06/2026 | 0 | 3059 |
| `c263e8e` | 22/06/2026 | 579 | 587 |
| `345ddc9` | 20/07/2026 | 737 | 692 |
| `31c5a39` | 11/08/2026 | 1 | 1 |
| `2d0260d` | 23/08/2026 | 1690 | 133 |
| `6693b53` | 23/08/2026 | 155 | 7 |
| `7c412f2` | 23/08/2026 | 0 | 57 |
| `225aa01` | 24/08/2026 | 11 | 2 |

# 6. Análisis de la evolución

## 6.1 Del prototipo a la estructura

El prototipo inicial concentraba todo el frontend en HTML (611 LOC en un solo archivo) y duplicaba las páginas en dos carpetas. Al separar HTML, CSS y JavaScript y pasar a varias páginas, el archivo más extenso bajó de 611 a 97 LOC y el tamaño medio por archivo de 129 a 38 LOC. El código total bajó de 1808 a 642 LOC al eliminar las duplicaciones, lo que muestra una mejora de modularidad y mantenibilidad sin pérdida de funcionalidad en la interfaz (H1 a H3).

## 6.2 Del frontend al sistema completo

El mayor salto del proyecto hasta ese momento ocurrió el 23 de agosto. El código pasó de 642 a 1906 LOC (3,0 veces), las funciones de 18 a 73 y los puntos de decisión de 29 a 179. Aparecieron la API PHP con 5 endpoints, 4 modelos y la base de datos con 4 tablas. La complejidad media por función subió de 2,61 a 3,45, un aumento esperable porque en esta etapa se incorporaron la validación de permisos por rol y el acceso a datos (H3 a H4).

## 6.3 Cierre del desarrollo

La versión del 24 de agosto agregó préstamos y dashboard: 112 LOC de PHP más, 2 endpoints más y 22 puntos de decisión más. El tamaño medio por archivo (64 y 63 LOC) y el archivo más extenso (140 LOC) casi no cambiaron, lo que indica que el crecimiento se hizo agregando módulos y no engordando los existentes (H4 a H5).

## 6.4 La versión actual

La versión actual pasó de 2021 a 2981 LOC (un 47 % más), de 73 a 140 funciones y de 201 a 384 puntos de decisión. El crecimiento se repartió entre el frontend (CSS de 350 a 622 LOC y JavaScript de 581 a 994 LOC) y el backend (PHP de 636 a 803 LOC y SQL de 62 a 143 LOC). Las tablas pasaron de 4 a 6 con la incorporación de las ubicaciones y los préstamos en el esquema, y la API sumó funciones de ubicaciones, asignación de técnicos, cambio de prioridad y eliminación de registros. La complejidad media por función se

mantuvo (3,75 y 3,74), pero la densidad de comentarios bajó del 5,2 % al 3,4 % y el archivo más extenso pasó de 140 a 308 LOC por la nueva hoja de estilos `layout.css` (H5 a H6).

### 6.5 Documentación y estabilidad

La densidad de comentarios pasó del 1,0 % al 5,2 % hasta H5 y se mantuvo en un rango estable (4,5 % a 6,0 %) desde que existe JavaScript, hasta bajar al 3,4 % en la versión actual. En cuanto a la estabilidad, entre H3 y H4 pasaron 34 días con un solo cambio mínimo (el commit del 11 de agosto, de una línea). Luego casi todo el desarrollo funcional se concentró en dos días (23 y 24 de agosto), con 1690 líneas insertadas en un solo commit.

## 7. Hallazgos y recomendaciones

1. **No hay pruebas automatizadas.** El repositorio no tiene archivos de pruebas, por lo que la cobertura medida es 0 %. Con 7 endpoints, 33 operaciones y 384 puntos de decisión, conviene agregar al menos pruebas de los modelos PHP y de las reglas de acceso por rol.
2. **Complejidad concentrada.** `inventario.js`, `incidencias.js`, `servicio.js`, `prestamos.js` y `api/prestamos.php` reúnen 175 de los 384 puntos de decisión (46 %). Son los candidatos para refactorizar si siguen creciendo.
3. **Menos comentarios en la versión actual.** La densidad de comentarios bajó del 5,2 % al 3,4 % mientras el código crecía un 47 %. Conviene documentar las funciones nuevas, en especial las de ubicaciones, asignación y préstamos.
4. **Historial poco granular.** Hay commits grandes (1690 líneas insertadas en uno solo) y mensajes genéricos ("commit 2", "commit 3"), y la versión actual llegó sin historial. Adoptar commits más chicos y descriptivos mejoraría la trazabilidad y permitiría medir la evolución con más resolución.
5. **Un solo contribuyente en el historial.** Los 9 commits figuran a nombre de `nicolas` (8) y `nico` (1). El historial de git no refleja el trabajo de los

cuatro integrantes del grupo, por lo que las métricas de actividad no sirven para repartir contribuciones.

6. **No hay registro de horas.** El esfuerzo real no se midió, por lo que las estimaciones no pueden contrastarse con datos propios. Conviene registrar las horas por tarea en Trello en las próximas etapas.
7. **Análisis por puntos de función no sincronizado con el código.** El análisis deja fuera cinco operaciones que existen en el código, entre 15 y 18 PF, y todavía marca como pendientes funciones que ya están implementadas. Conviene mantenerlo actualizado con cada versión.
8. **Medición con herramientas.** Las cifras se obtuvieron con análisis de texto. Para la entrega final conviene contrastarlas con una herramienta de análisis estático (por ejemplo, SonarQube o PHPMetrics) y reportar las diferencias.

# 8. Referencias Bibliográficas

Las referencias se presentan conforme a las normas APA (7.ª edición), respetando los criterios de citación y formato establecidos por dicha normativa.

Albrecht, A. J. (1979). Measuring application development productivity. En *Proceedings of the IBM Applications Development Symposium* (pp. 83–92). IBM.

Boehm, B. W. (1981). *Software engineering economics*. Prentice Hall.

International Function Point Users Group. (2010). *Function point counting practices manual* (Release 4.3.1). IFPUG.

International Organization for Standardization & International Electrotechnical Commission. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC 25010:2011).

McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering, SE-2*(4), 308–320.

Quantitative Software Management. (s. f.). *Function point languages table* (Version 5.0). https://qsm.com/FPGearing.html

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.